Toward Robust Machine Learning based Network Intrusion Detection Systems

Omar Anser

Inria, Université de Lorraine, CNRS, LORIA, Nancy, France (currently post doc Univ. Luxembourg)

President

Bernardetta Addis
Professeure

Université de Lorraine

Rapporteurs

Nadjib Aït-Saadi
Professeur des universités

UVSQ - Université Paris-Saclay

Francesco Bronzino
Maître de conférences HDR

ENS de Lyon

Examiner

Grégory Blanc
Maître de conférences

Télécom SudParis

Supervisors

Isabelle Chrisment
Professeure

Université de Lorraine

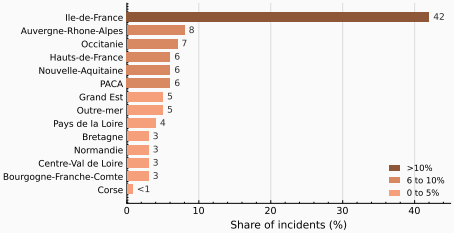
Jérôme François
Research scientist

University of Luxembourg, SnT

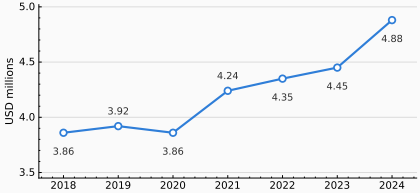
Introduction

Why this topic matters

- Modern societies and public services increasingly rely on computer networks $\longrightarrow$ Higher exposure to cyber incidents.
- ANSSI handled **4,386** security events in 2024 (**+15%** vs. 2023), including **1,361** confirmed incidents, i.e., nearly **4** per day.
- **$4.88M** average global cost of a data breach in 2024.
- **35%** of reported cyber incidents in the French health sector affected patient care.



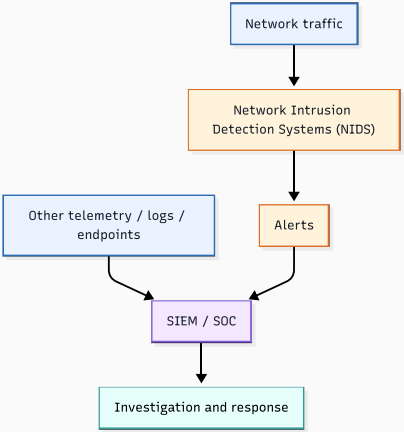
Source: **ANSSI, Panorama de la cybermenace 2024**



Source: **IBM Security, Cost of a Data Breach Report 2024**

Where NIDS fits in cyber defense

- Continuous monitoring to manage cyber risk
- SOCs and SIEM platforms centralize this monitoring
- Network traffic gives direct visibility into system communications
- NIDS turn traffic into alerts for investigation and response[2]



⇒ **NIDS is one analytical layer inside the broader monitoring and response chain[1].**

Refs: [1] NIST SP 800-86 (2006); [2] Scarfone & Mell, Guide to Intrusion Detection and Prevention Systems (2007).

From signatures to ML-based NIDS

Rule-based NIDS

- Tools: **Snort**[1], **Suricata**[2], **Zeek**[3]
- Detect only known attack patterns or protocol violations, leaving unknown ones undetected.
- Encrypted traffic limits payload-based rules[5]

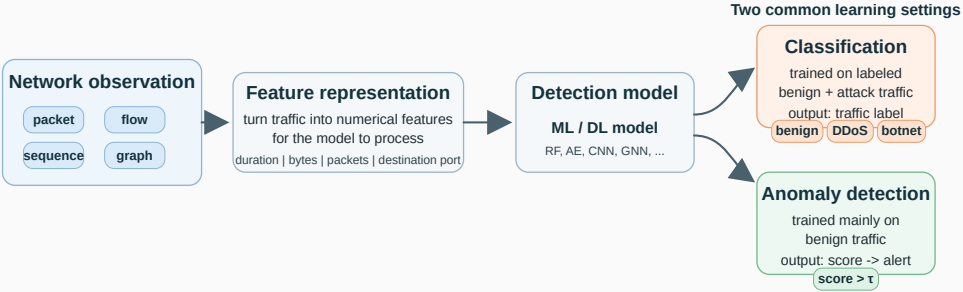
ML-based NIDS[4]

- Example: **Random Forest (RF) on flows**
- Learn traffic patterns from historical data to generalize beyond exact rules.
- Can learn complex traffic patterns even under encryption[5]

Operational reality: mostly hybrid[6]**.** E.g. SOCs use **Suricata/Zeek** for known alerts + **ML on flows** to flag suspicious HTTPS C&C behavior.

Refs: [1] Snort [Roesch, 1999]; [2] Suricata (OISF); [3] Zeek [Paxson, 1999]; [4] ML for NIDS [Sommer & Paxson, 2010]; [5] encrypted traffic [Anderson & McGrew, 2016]; [6] SANS Detection & Response Survey (2024).

ML-based NIDS as a prediction task

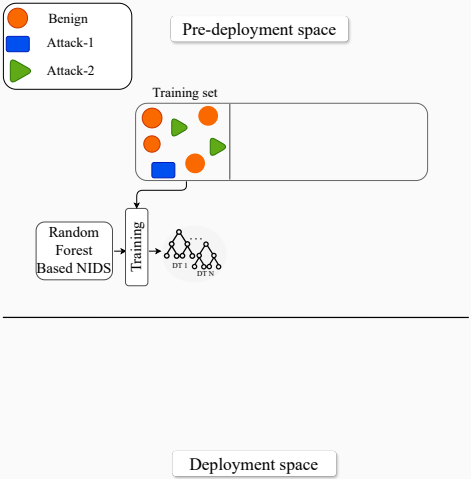


$\Rightarrow$ **This common ML setting leads to the standard train $\rightarrow$ validate $\rightarrow$ test $\rightarrow$ deploy workflow[1][2].**

Refs: [1] Sommer & Paxson (2010); [2] Buczak & Guven (2016).

ML-based NIDS Workflow: Train → Validate → Test → Deploy

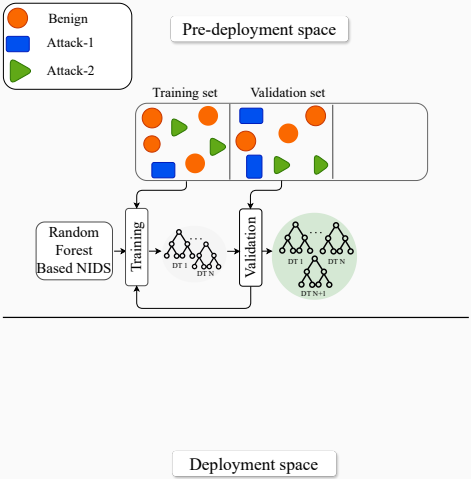
- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.



ML-based NIDS Workflow: Train → Validate → Test → Deploy

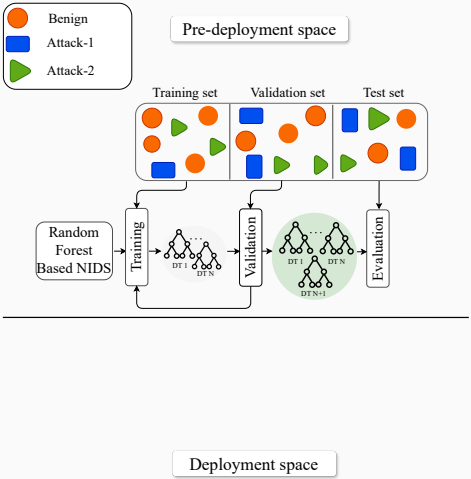
- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters** and, in anomaly detection, the **threshold**).

Hyperparameters: settings chosen before training that influence model behavior.
Example: In RF, too few trees can give unstable predictions, while too many can increase computation and overfit the training data.



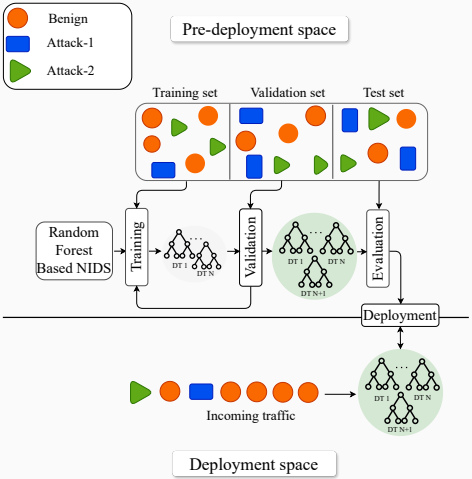
ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters** and, in anomaly detection, the **threshold**).
- A final **test** set, $\mathcal{D}_{\text{test}}$, measures performance on previously unseen data.



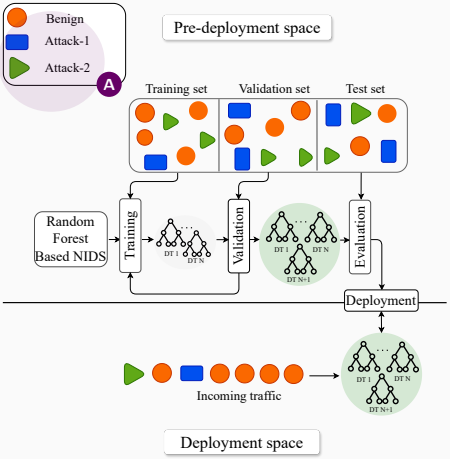
ML-based NIDS Workflow: Train → Validate → Test → Deploy

- The model is trained on a labelled **training** set $\mathcal{D}_{\text{train}}$.
- A separate **validation** split $\mathcal{D}_{\text{val}}$ is used to refine the model **configuration** (**hyperparameters** and, in anomaly detection, the **threshold**).
- A final **test** set, $\mathcal{D}_{\text{test}}$, measures performance on previously unseen data.
- The selected model is then **deployed** to monitor live traffic.



How ML has been used to improve NIDS

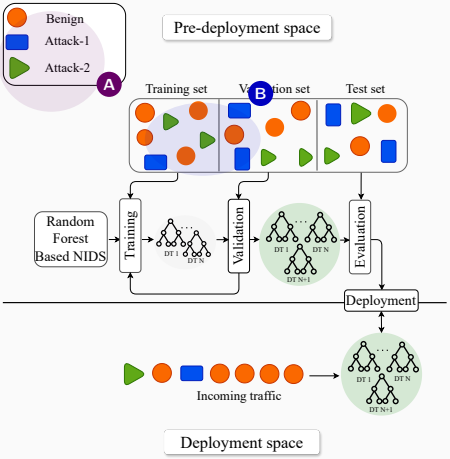
- **A** Richer traffic representations ⟶ Flow-MAE[1], NetVigil[2]
  - e.g., packet, flow, sequence, graph
- **B** Better use of limited labels ⟶ Active-SVDD[3], INSOMNIA[4]
  - e.g., active learning, semi-supervised learning, and pseudo-labeling
- **C** Better detection performance ⟶ BotMiner[5], Kitsune[6]
  - e.g., DDoS, botnet, web attacks
- **D** More practical deployment ⟶ AOC-IDS[7], ENIDrift[8]
  - e.g., lightweight, online, real-time detection
- **E** Better robustness in practice ⟶ CADE[9], OWAD[10]
  - e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation



Refs: [1] Flow-MAE [Hang et al., 2023]; [2] NetVigil [Hsieh et al., 2024]; [3] Active-SVDD [Gornitz et al., 2009]; [4] INSOMNIA [Pierazzi et al., 2021]; [5] BotMiner [Gu et al., 2008]; [6] Kitsune [Mirsky et al., 2018]; [7] AOC-IDS [Zhao et al., 2024]; [8] ENIDrift [Wang, 2022]; [9] CADE [Yang et al., 2021]; [10] OWAD [Hu et al., 2023].

How ML has been used to improve NIDS

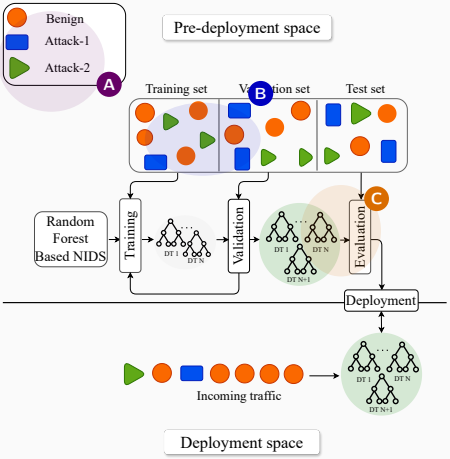
- **A** Richer traffic representations ⟶ Flow-MAE[1], NetVigil[2]
  - e.g., packet, flow, sequence, graph
- **B** Better use of limited labels ⟶ Active-SVDD[3], INSOMNIA[4]
  - e.g., active learning, semi-supervised learning, and pseudo-labeling
- **C** Better detection performance ⟶ BotMiner[5], Kitsune[6]
  - e.g., DDoS, botnet, web attacks
- **D** More practical deployment ⟶ AOC-IDS[7], ENIDrift[8]
  - e.g., lightweight, online, real-time detection
- **E** Better robustness in practice ⟶ CADE[9], OWAD[10]
  - e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation



Refs: [1] Flow-MAE [Hang et al., 2023]; [2] NetVigil [Hsieh et al., 2024]; [3] Active-SVDD [Gornitz et al., 2009]; [4] INSOMNIA [Pierazzi et al., 2021]; [5] BotMiner [Gu et al., 2008]; [6] Kitsune [Mirsky et al., 2018]; [7] AOC-IDS [Zhao et al., 2024]; [8] ENIDrift [Wang, 2022]; [9] CADE [Yang et al., 2021]; [10] OWAD [Hu et al., 2023].

How ML has been used to improve NIDS

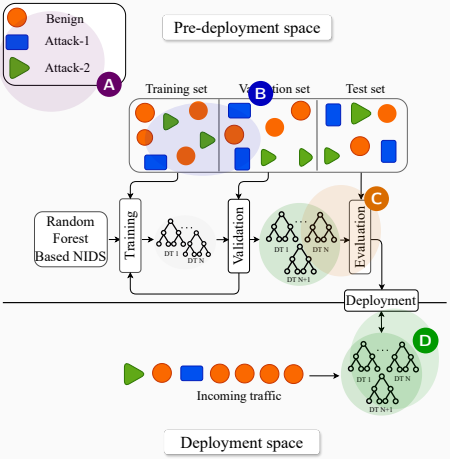
- **A** Richer traffic representations ⟶ Flow-MAE[1], NetVigil[2]
  - e.g., packet, flow, sequence, graph
- **B** Better use of limited labels ⟶ Active-SVDD[3], INSOMNIA[4]
  - e.g., active learning, semi-supervised learning, and pseudo-labeling
- **C** Better detection performance ⟶ BotMiner[5], Kitsune[6]
  - e.g., DDoS, botnet, web attacks
- **D** More practical deployment ⟶ AOC-IDS[7], ENIDrift[8]
  - e.g., lightweight, online, real-time detection
- **E** Better robustness in practice ⟶ CADE[9], OWAD[10]
  - e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation



Refs: [1] Flow-MAE [Hang et al., 2023]; [2] NetVigil [Hsieh et al., 2024]; [3] Active-SVDD [Gornitz et al., 2009]; [4] INSOMNIA [Pierazzi et al., 2021]; [5] BotMiner [Gu et al., 2008]; [6] Kitsune [Mirsky et al., 2018]; [7] AOC-IDS [Zhao et al., 2024]; [8] ENIDrift [Wang, 2022]; [9] CADE [Yang et al., 2021]; [10] OWAD [Hu et al., 2023].

How ML has been used to improve NIDS

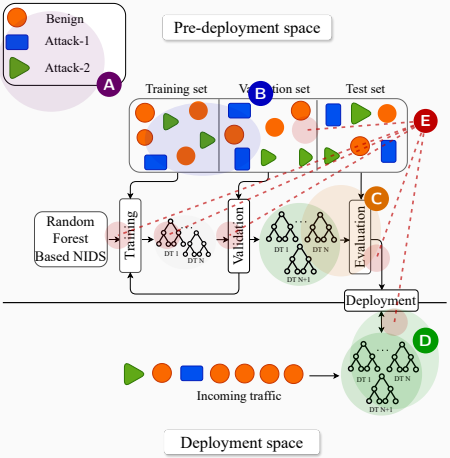
- **A** Richer traffic representations ⟶ Flow-MAE[1], NetVigil[2]
  - e.g., packet, flow, sequence, graph
- **B** Better use of limited labels ⟶ Active-SVDD[3], INSOMNIA[4]
  - e.g., active learning, semi-supervised learning, and pseudo-labeling
- **C** Better detection performance ⟶ BotMiner[5], Kitsune[6]
  - e.g., DDoS, botnet, web attacks
- **D** More practical deployment ⟶ AOC-IDS[7], ENIDrift[8]
  - e.g., lightweight, online, real-time detection
- **E** Better robustness in practice ⟶ CADE[9], OWAD[10]
  - e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation



Refs: [1] Flow-MAE [Hang et al., 2023]; [2] NetVigil [Hsieh et al., 2024]; [3] Active-SVDD [Gornitz et al., 2009]; [4] INSOMNIA [Pierazzi et al., 2021]; [5] BotMiner [Gu et al., 2008]; [6] Kitsune [Mirsky et al., 2018]; [7] AOC-IDS [Zhao et al., 2024]; [8] ENIDrift [Wang, 2022]; [9] CADE [Yang et al., 2021]; [10] OWAD [Hu et al., 2023].

How ML has been used to improve NIDS

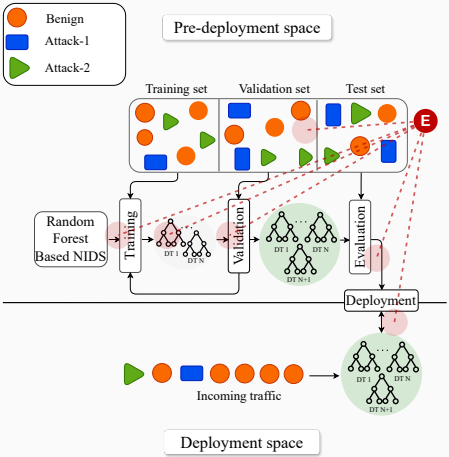
- **A** Richer traffic representations ⟶ Flow-MAE[1], NetVigil[2]
  - e.g., packet, flow, sequence, graph
- **B** Better use of limited labels ⟶ Active-SVDD[3], INSOMNIA[4]
  - e.g., active learning, semi-supervised learning, and pseudo-labeling
- **C** Better detection performance ⟶ BotMiner[5], Kitsune[6]
  - e.g., DDoS, botnet, web attacks
- **D** More practical deployment ⟶ AOC-IDS[7], ENIDrift[8]
  - e.g., lightweight, online, real-time detection
- **E** Better robustness in practice ⟶ CADE[9], OWAD[10]
  - e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation



Refs: [1] Flow-MAE [Hang et al., 2023]; [2] NetVigil [Hsieh et al., 2024]; [3] Active-SVDD [Gornitz et al., 2009]; [4] INSOMNIA [Pierazzi et al., 2021]; [5] BotMiner [Gu et al., 2008]; [6] Kitsune [Mirsky et al., 2018]; [7] AOC-IDS [Zhao et al., 2024]; [8] ENIDrift [Wang, 2022]; [9] CADE [Yang et al., 2021]; [10] OWAD [Hu et al., 2023].

How ML has been used to improve NIDS

- A Richer traffic representations ⟶ Flow-MAE, NetVigil
  - e.g., packet, flow, sequence, graph
- B Better use of limited labels ⟶ Active-SVDD, INSOMNIA
  - e.g., active learning, semi-supervised learning, and pseudo-labeling
- C Better detection performance ⟶ BotMiner, Kitsune
  - e.g., DDoS, botnet, web attacks
- D More practical deployment ⟶ AOC-IDS, ENIDrift
  - e.g., lightweight, online, real-time detection
- E Better robustness in practice ⟶ CADE[1], OWAD[2]
  - e.g., adaptability to evolving traffic, resilience to adversarial attacks, and more challenging evaluation



⇒ **The other axes optimize the detector inside a given setup, robustness questions the validity of that setup itself.**

Refs: [1] CADE [Yang et al., 2021]; [2] OWAD [Hu et al., 2023].

Focus of this thesis: robustness in ML-based NIDS

Robustness in this thesis

- One fixed benchmark split is not enough.
- Robustness in operation: effectiveness under evolving traffic and attacks.
- Robustness in assessment: evaluation that truly stresses the detector.

$\Rightarrow$ **Robustness = adaptation + rigorous evaluation.**

Roadmap of the presentation

Part I Adaptation

- Post-deployment robustness
- Fast NIDS reconfiguration
- Meta-learning-based auto-configuration
- Context change handling

Part II Evaluation

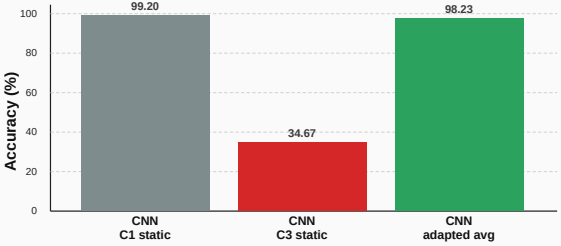
- Test-set difficulty assessment → relvealing NIDS weaknesses
- Targeted augmentation of the test set when difficulty is low

***Publications*: Part I – *Auto-tuning of Hyper-parameters for Detecting Network Intrusions via Meta-learning* (NOMS 2023, AnNet);**
***Automated Machine Learning Configuration to Learn Intrusion Detectors on Attack-Free Datasets* (LCN 2024).**
Part II – *TATA: Benchmark NIDS Test Sets Assessment and Targeted Augmentation* (ESORICS 2025);
***An Empirical Study on Configuration Robustness of Unsupervised IDS* (NOMS 2026).**

Part I: Adaptation

Why ML-based NIDS still need explicit adaptation

- Historical traffic for training; static detector at deployment.
- Assumes training and deployment traffic are statistically identical.
- Benign traffic evolves, attackers adapt → the network context changes.

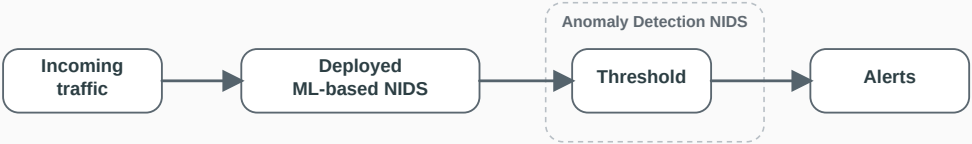


Source: Qiao et al., IEEE TII 2022, Table V/VI and Section IV-C

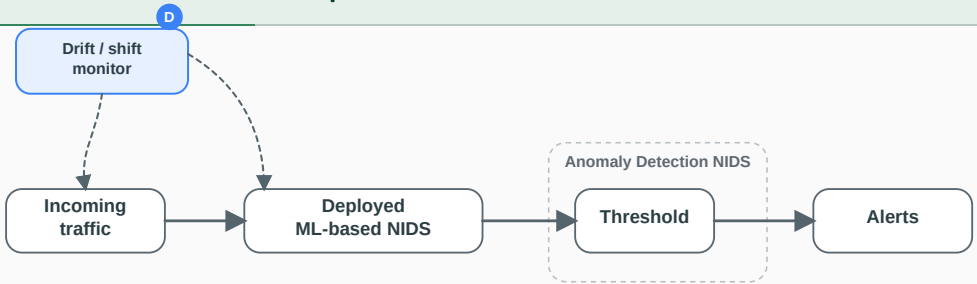
⇒ **Detection performance can degrade** ⇒ **explicit adaptation mechanisms are needed**

Adaptation = post-deployment update of (data | model | config | threshold) to maintain performance as the network context changes.

State of the art: main adaptation families

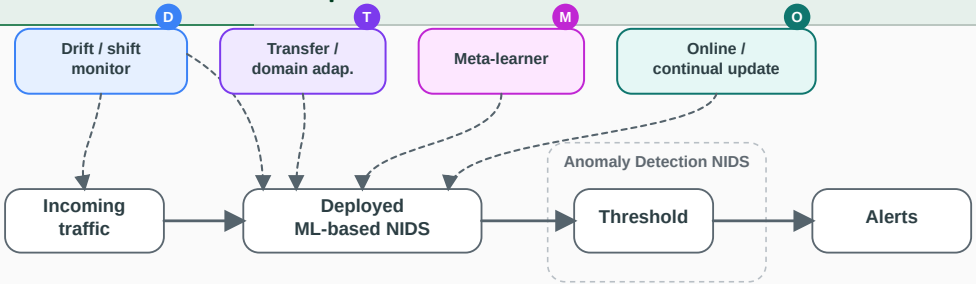


State of the art: main adaptation families



D **Drift detection**
monitor change in traffic or detector scores
CADE, OWAD, TRANSCEND

State of the art: main adaptation families



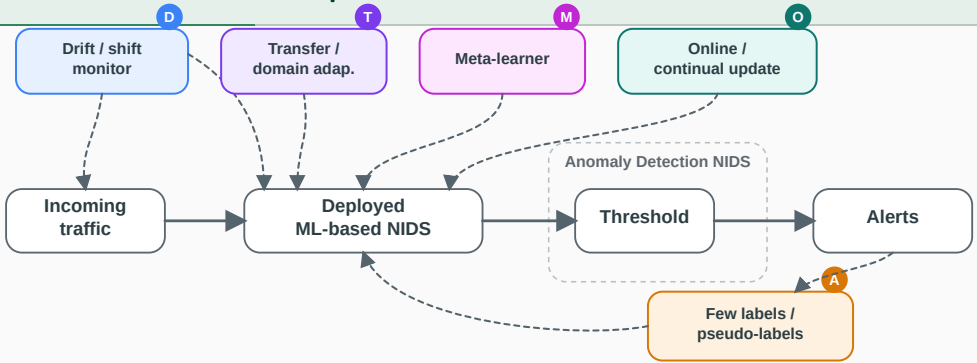
D Drift detection
monitor change in traffic or detector scores
CADE, OWAD, TRANSCEND

T Transfer / domain adap.
reuse knowledge from another context
ADA, NAEF, DI-NIDS

M Meta-learning
learn how to adapt quickly from prior tasks
FC-Net, PTN-IDS, RETSINA

O Online / continual learning
update model weights over time
AOC-IDS, ENIDrift, ReCDA

State of the art: main adaptation families



D Drift detection
monitor change in traffic or detector scores
CADE, OWAD, TRANSCEND

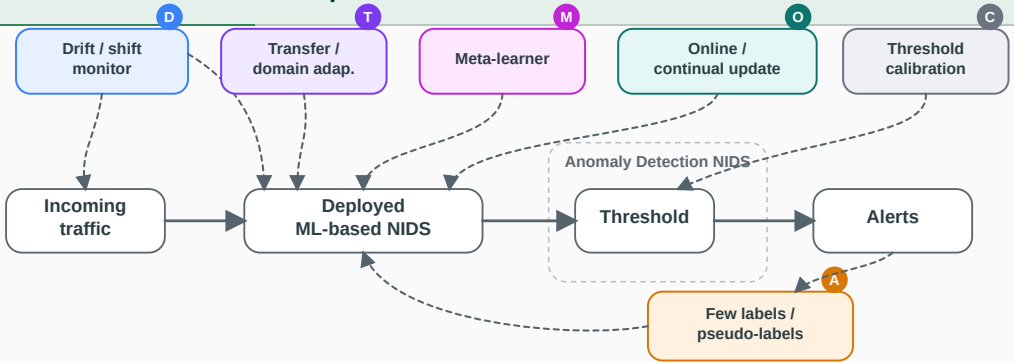
T Transfer / domain adap.
reuse knowledge from another context
ADA, NAEF, DI-NIDS

M Meta-learning
learn how to adapt quickly from prior tasks
FC-Net, PTN-IDS, RETSINA

O Online / continual learning
update model weights over time
AOC-IDS, ENIDrift, ReCDA

A Active / semi-supervised
use few labels or pseudo-labels
Active-SVDD, HITL-IDS, METALS

State of the art: main adaptation families



D Drift detection
monitor change in traffic or detector scores
CADE, OWAD, TRANSCEND

T Transfer / domain adap.
reuse knowledge from another context
ADA, NAEF, DI-NIDS

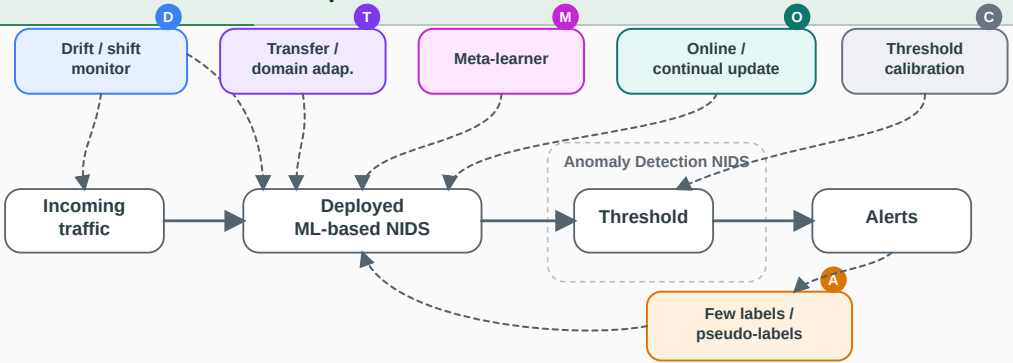
M Meta-learning
learn how to adapt quickly from prior tasks
FC-Net, PTN-IDS, RETSINA

O Online / continual learning
update model weights over time
AOC-IDS, ENIDrift, ReCDA

A Active / semi-supervised
use few labels or pseudo-labels
Active-SVDD, HITL-IDS, METALS

C Threshold calibration
adjust alert cutoff after training
Bridges, Hundman, MAGPIE

State of the art: main adaptation families



D Drift detection
monitor change in traffic or detector scores
CADE, OWAD, TRANSCEND

T Transfer / domain adap.
reuse knowledge from another context
ADA, NAEF, DI-NIDS

M Meta-learning
learn how to adapt quickly from prior tasks
FC-Net, PTN-IDS, RETSINA

O Online / continual learning
update model weights over time
AOC-IDS, ENIDrift, ReCDA

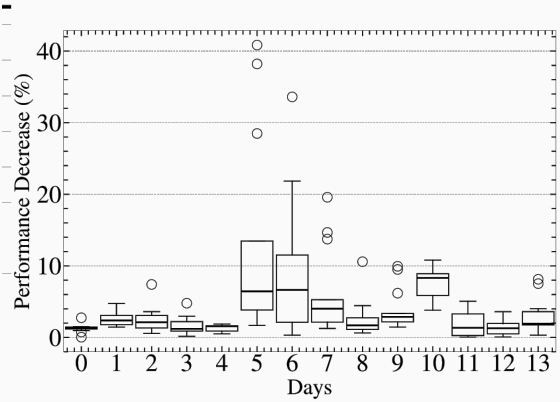
A Active / semi-supervised
use few labels or pseudo-labels
Active-SVDD, HITL-IDS, METALS

C Threshold calibration
adjust alert cutoff after training
Bridges, Hundman, MAGPIE

$\Rightarrow$ **Most methods adapt model, or threshold; configuration remains untargeted.**

Is hyperparameter configuration important?

- Can a day-t optimal config be reused on day t'?
- Each boxplot $\rightarrow$ performance drop from transferring another day's best config.
- Limited loss on some days; 20–40% degradation on others.



Source: Anser et al., IEEE/IFIP NOMS 2026, Figure 5

$\Longrightarrow$ **Configuration transfer helps in some contexts, but is not reliable in general.**

How is configuration handled today?

Methods

- Grid search, random search[1], Bayesian optimization (BO)[2], evolutionary search.
- Repeated train-and-validate over candidate configurations.

Main drawback

- Offline and expensive.
- Restarted from scratch after each context change.

Real NIDS example[3]

- Bayesian optimization + binary Random Forest NIDS.

300	10 s	$\approx$ 50 min
iterations	per iteration	per daily context

$\Longrightarrow$ **Too slow for repeated post-deployment reconfiguration.**

Research question (RQ1)

Research question

How can we find a configuration of a ML-based NIDS in a timely manner while maintaining an acceptable level of detection performance?

From our analysis of hyperparameter impact

- A NIDS configuration method has to be applied at each network context change.
- A new network context is defined by a new attack in the network or by a topology change.

Source: Anser et al., *An Empirical Study on Configuration Robustness of Unsupervised IDS*, IEEE/IFIP NOMS 2026.

$\Rightarrow$ **Auto-configuration via meta-learning**

Auto-configuration via Meta-learning

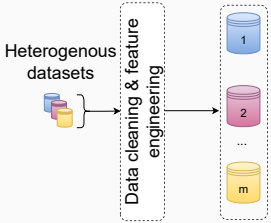
A. Meta-modeling

B. On the fly NIDS configuration

Trained NIDS

Auto-configuration via Meta-learning

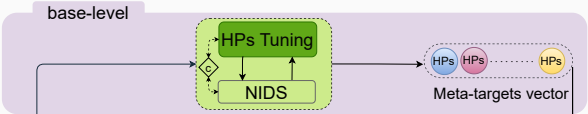
A. Meta-modeling



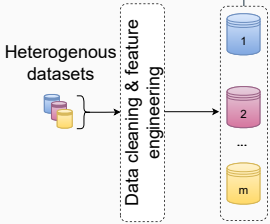
B. On the fly NIDS configuration

Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold



A. Meta-modeling

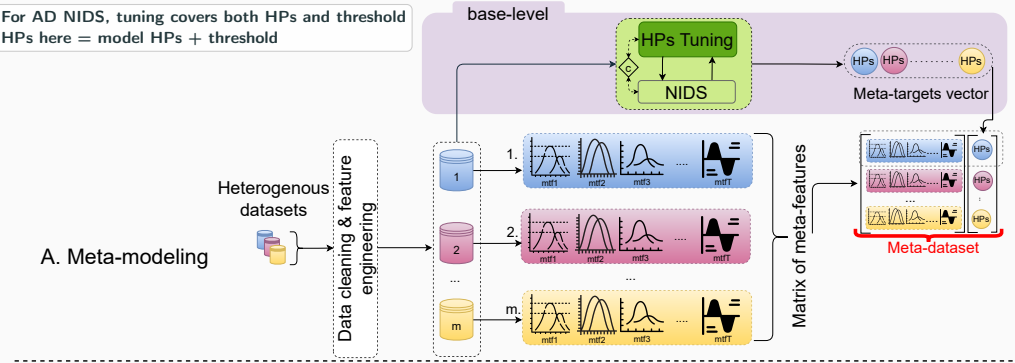


B. On the fly NIDS configuration

Trained NIDS

Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold

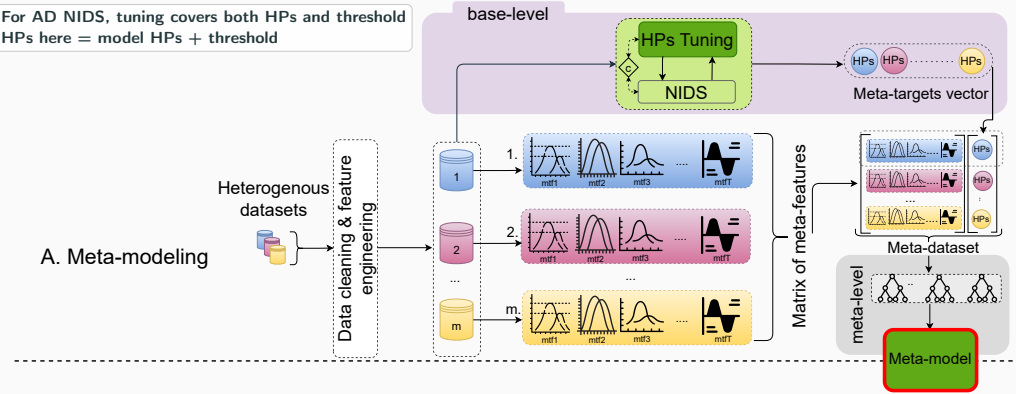


B. On the fly NIDS configuration

Trained NIDS

Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold

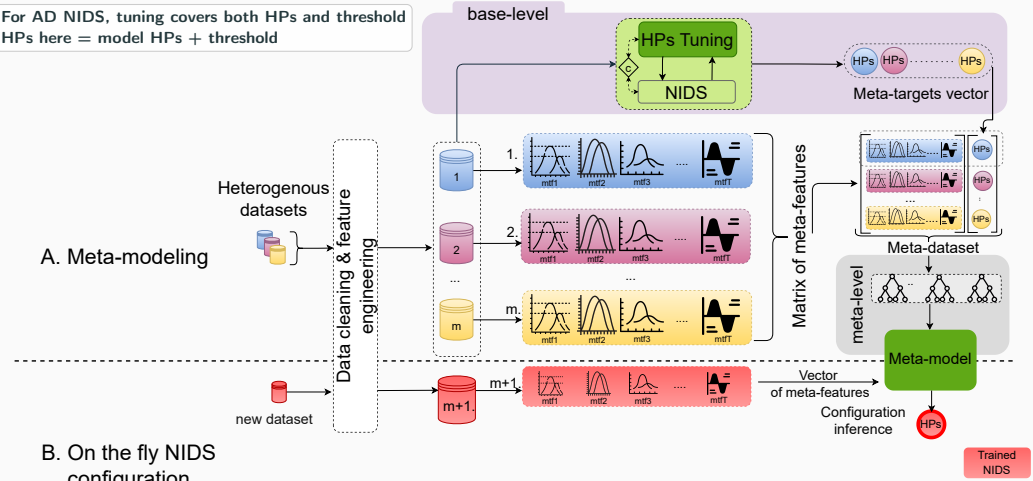


B. On the fly NIDS configuration

Trained NIDS

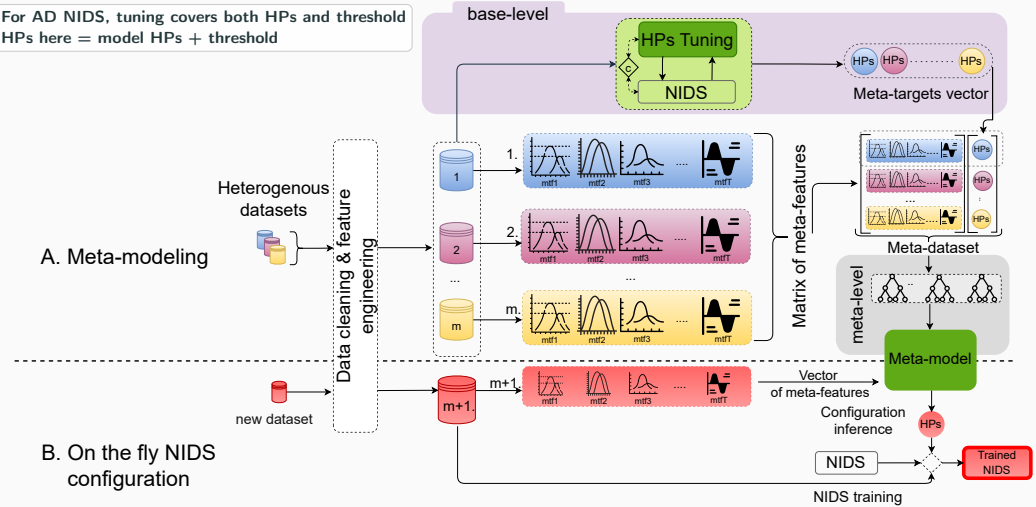
Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold



Auto-configuration via Meta-learning

For AD NIDS, tuning covers both HPs and threshold
HPs here = model HPs + threshold



Methodology applied in two settings

Supervised setting

- Binary Random Forest on labeled benign/attack traffic.
- Reference: Anser et al., **NOMS 2023** (AnNet workshop).
- Contexts: **17 daily** IDS2017/IDS2018 datasets.
- Search space: **2 RF HPs** (‘max_features‘, ‘min_samples_leaf‘).
- BO baseline: **300 iterations/day**, optimized on **MCC**.

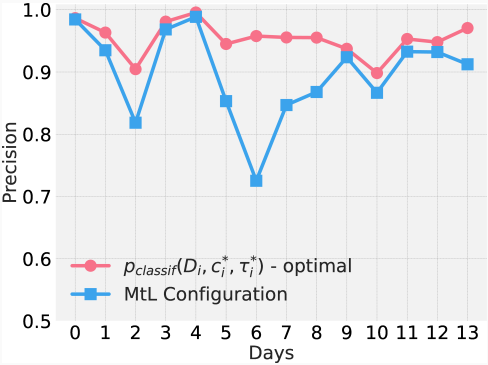
Unsupervised setting

- Isolation Forest-based AD IDS trained on attack-free data.
- Reference: Anser et al., **LCN 2024** (short paper).
- Contexts: **14 daily** IDS2017/IDS2018 datasets.
- Search space: **7 iForest HPs + threshold**.
- BO baseline: offline daily optimum with labels; metric: **precision**.

⇒ **Same meta-learning pipeline, but different detectors, search spaces, and targets.**

Results: precision

- High overall precision in attack-free AD.
- Higher variability on days **2** and **5–8**.
- Harder contexts → weaker transfer.
- Supervised setting: meta-learning close to the (sub-)optimal configuration.

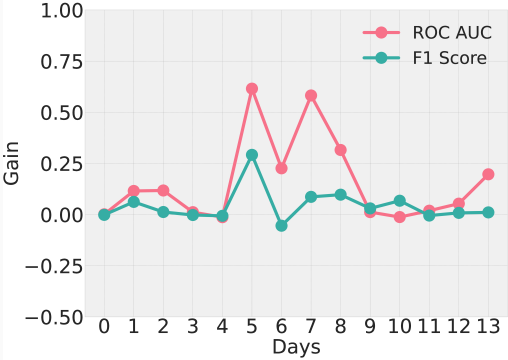


Unsupervised setting

⇒ **High precision overall; attack-free AD reveals the harder transfer days.**

Results: gain

- Attack-free AD: meta-learning can beat budget-matched BO.
- Strongest gains on harder days, especially **5–8**.
- Limited negative gains at equal cost.
- Supervised setting: large gains over the default.

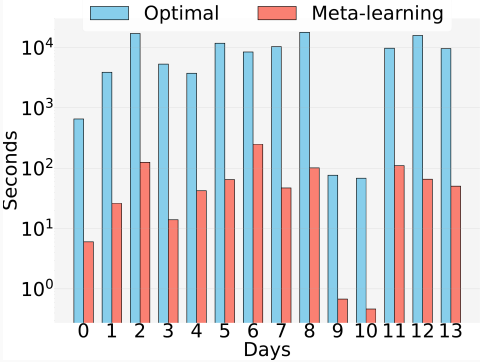


Unsupervised setting

⇒ **Cheaper than BO, yet competitive or better at equal budget.**

Results: configuration time

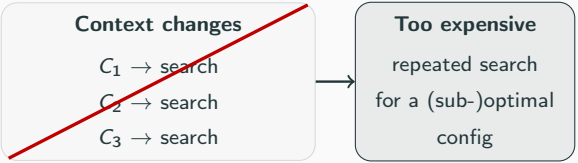
- Attack-free AD: seconds-to-minutes reconfiguration.
- BO runtime can exceed **100,000 s**.
- About **160x** faster overall.
- Supervised setting: about **9x** faster than finding a (sub-)optimal configuration.



Unsupervised setting

⇒ **Time advantage becomes decisive for online reconfiguration.**

Part I conclusion



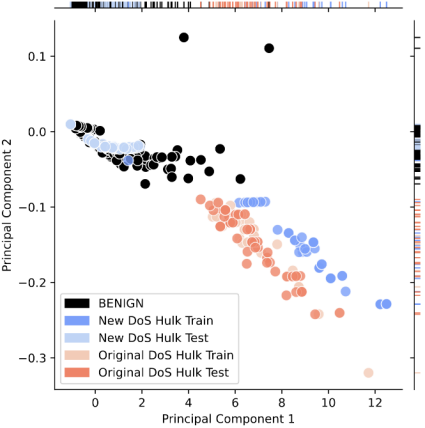
- Meta-learning preserves strong detection performance.
- The operational benefit is time: visible in supervised NIDS and decisive in attack-free AD.

$\Rightarrow$ **A comprehensive analysis of the impact of configuration not presented here is available in our NOMS 2026 paper.**

Part II: Evaluation

Benchmark NIDS datasets — A Closer Look

- A single source dataset partitioned into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$.
- Reported evaluation accuracies $>$ **90%** in published works on public benchmarks (IDS2017, UNSW-NB15...) $\rightarrow$ solved problem or benchmark-induced illusions?
- A near-complete overlap of DoS-Hulk flows in IDS2017's $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$[1].

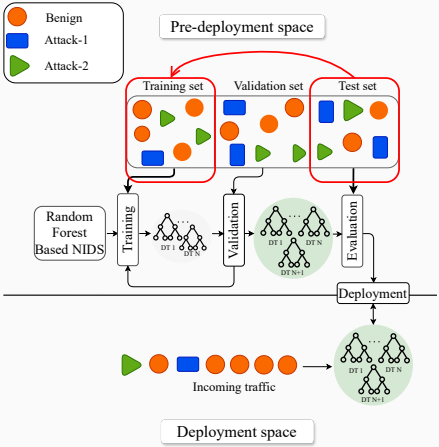


Overlap between $\mathcal{D}_{\text{test}}$ and $\mathcal{D}_{\text{train}}$ does not prevent overfitting $\rightarrow$ it can hide it.

[1] Flood et al., Bad Design Smells in Benchmark NIDS Datasets, IEEE EuroS&P 2024.

Focusing on $\mathcal{D}_{\text{test}}$ and its relation to $\mathcal{D}_{\text{train}}$

- A robust evaluation should expose challenging cases, including corner cases.
- Pre-deployment evaluation mainly relies on the test set $\mathcal{D}_{\text{test}}$.
- The key question is whether $\mathcal{D}_{\text{test}}$ really stresses the ML-based NIDS.
- The detector is constructed from $\mathcal{D}_{\text{train}} \rightarrow \mathcal{D}_{\text{test}}$ must be assessed relative to $\mathcal{D}_{\text{train}}$.



$\Rightarrow$ **This part focuses on $\mathcal{D}_{\text{test}}$ and its relation to $\mathcal{D}_{\text{train}}$.**

RQ2 — Targeted Augmentation

Given a **too-easy** initial $\mathcal{D}_{\text{test}}$, can we apply systematic augmentation to generate new, **realistic**, and **challenging** traffic that is specifically designed to expose the hidden weaknesses of the NIDS?

Research Questions

RQ1 — Test-set Assessment

How can we create a numerical score or a set of metrics to objectively measure the difficulty of a $\mathcal{D}_{\text{test}}$ for a NIDS?

RQ2 — Targeted Augmentation

Given a **too-easy** initial $\mathcal{D}_{\text{test}}$, can we apply systematic augmentation to generate new, **realistic**, and **challenging** traffic that is specifically designed to expose the hidden weaknesses of the NIDS?

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage			

Neuron coverage: Fraction of neurons whose activation exceeds a preset threshold at least once across the test set; higher → explore more of the model's internal logic.

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage			
Surprise coverage			

Surprise coverage: Measures how broadly the test set differs from what the model saw in training; higher → the test set includes highly surprising inputs.

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage			
Surprise coverage			
Mutation testing			

Mutation testing: Mutates the model and/or its training setup (e.g., weights, architecture, or hyperparameters), then checks if the test set can distinguish the mutant from the original model.

Related Work

Method category	RQ1	RQ2	Key limitations
Neuron coverage	✓	✗	Model access (🔓) + NN-only ( ) + Ignores train–test alignment (📏)
Surprise coverage	✓	✗	🔓 + + Proximity-only focus (🎯)
Mutation testing	✓	✓	🔓 + 📏 + Model-change sensitive (🛠) Retrain per dataset (🔄) + Synthetic data (🧪)
Manual (Flood et al., 2024)	✗	✓	Unguided augmentation (👆)

G1 Model-related gaps **G2** Shortcomings in assessing $\mathcal{D}_{\text{test}}$ → RQ1 **G3** Weaknesses in $\mathcal{D}_{\text{test}}$ augmentation workflows → RQ2

TATA (Test-set Assessment & Targeted Augmentation)

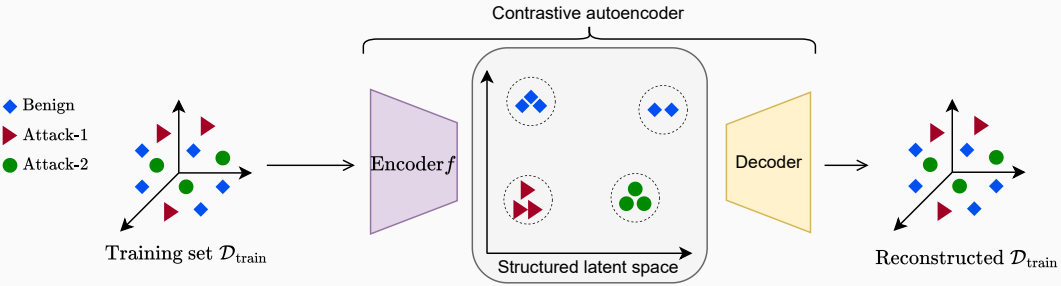
Goal: Close the gaps identified in related work and answer the research questions.

1. **Preliminary Phase** **G1**: + +
   Approximate $\mathcal{D}_{\text{train}}$'s decision boundary while being model agnostic

2. **Phase 1 – Assesses $\mathcal{D}_{\text{test}}$'s quality (RQ1)** **G2**: +
   Define three metrics (**Proximity**, **Diversity**, and **Scarcity**) to assess the **alignment** of $\mathcal{D}_{\text{test}}$ with respect to $\mathcal{D}_{\text{train}}$'s distribution.

3. **Phase 2 – $\mathcal{D}_{\text{test}}$ Augmentation (RQ2)** **G3**: + +
   Reinforcement learning (RL) guides a testbed to generate **realistic traffic** that fills uncovered metric gaps, producing **a more challenging** $\mathcal{D}_{\text{test}}$.

Preliminary Phase: Approximating $\mathcal{D}_{\text{train}}$'s decision boundary



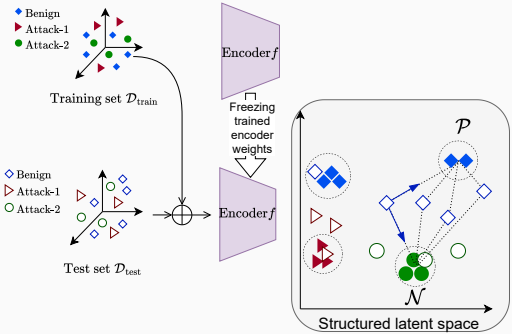
- Decision boundaries $\approx$ cluster borders in the latent space; these are captured in the encoder's weights.

Phase 1: Test-set Assessment

- Encode $\mathcal{D}_{\text{train}}$ & $\mathcal{D}_{\text{test}} \rightarrow \mathcal{Z}_{\text{test}}$ is located relative to $\mathcal{Z}_{\text{train}} \rightarrow$ we now work entirely in latent space $(\mathcal{Z}_{\text{train}}, \mathcal{Z}_{\text{test}})$.

- Proximity, diversity, and scarcity to characterize where $\mathcal{Z}_{\text{test}}$ lies in latent space relative to $\mathcal{Z}_{\text{train}}$ clusters.

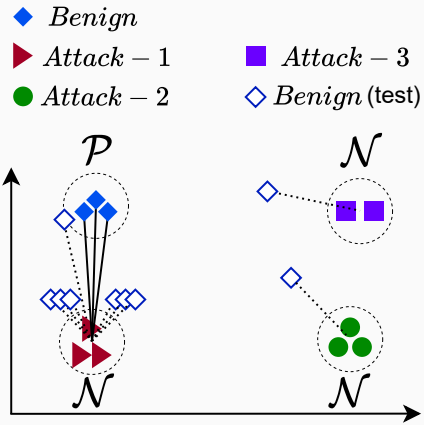
- For each test embedding z, we identify:
  - $P(z)$: the nearest **same-label cluster** (*positive cluster*)
  - $N(z)$: the nearest **different-label cluster** (*negative cluster*)



Proximity $P \in [0, 1]$, higher = nearer to different-label boundaries

- Quantifies $\mathcal{Z}_{\text{test}}$ closeness to $N(z)$; high proximity $\Rightarrow$ closer to a different label decision boundary $\Rightarrow$ harder cases.

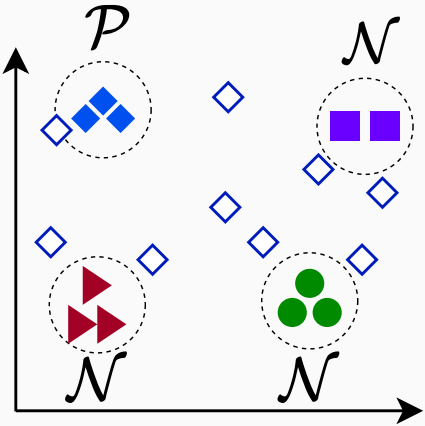
- P = **max** one-sided Kolmogorov–Smirnov statistic (over clusters): compares $\mathcal{Z}_{\text{test}}$ vs. $\mathcal{Z}_{\text{train}}$ distances to $N(z)$



Diversity $D \in [0, 1]$, higher = broader, less redundant coverage

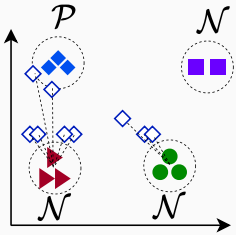
- Assesses the variety of $\mathcal{Z}_{\text{test}}$; higher diversity $\Rightarrow$ minimal redundancy $\Rightarrow$ broader coverage of the latent space.

- D = **Average** of normalized **Vendi Scores** (over clusters)

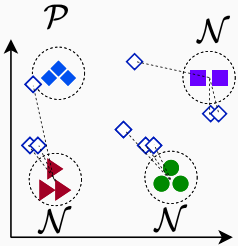


Scarcity $S \in [0, 1]$ higher = boundaries challenged more evenly

- Evaluates how uniformly $\mathcal{Z}_{\text{test}}$ are distributed among all potential negative clusters (i.e., decision boundaries).
- High scarcity $\Rightarrow$ a uniform spread $\Rightarrow$ all boundaries are exercised $\Rightarrow$ a more robust test set.
- S = **Average** of the **(1 - Gini coefficient)** (over clusters)

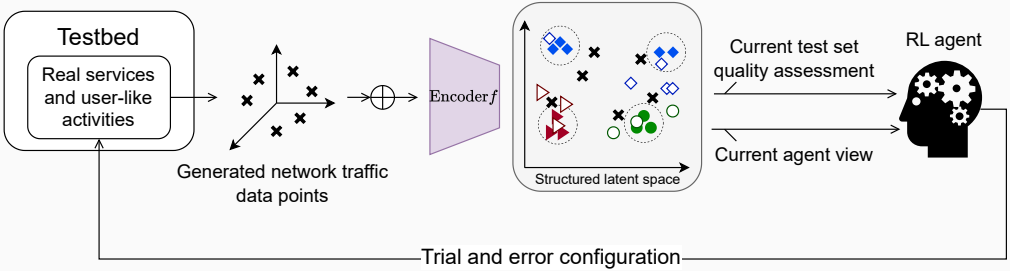


Bad scarcity



Optimal scarcity

Phase 2: Targeted Augmentation



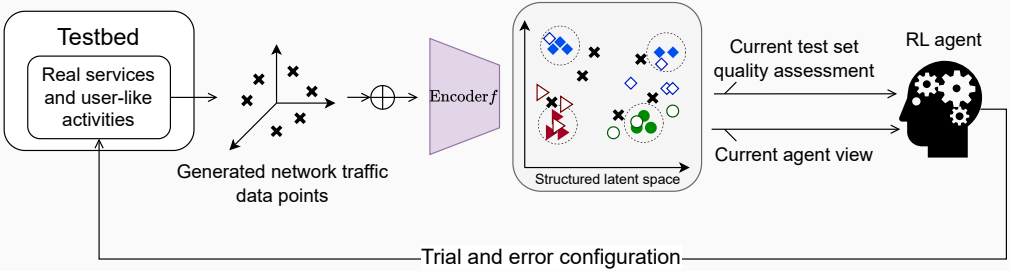
- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\boxed{\text{observe latent-space}} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$

$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation



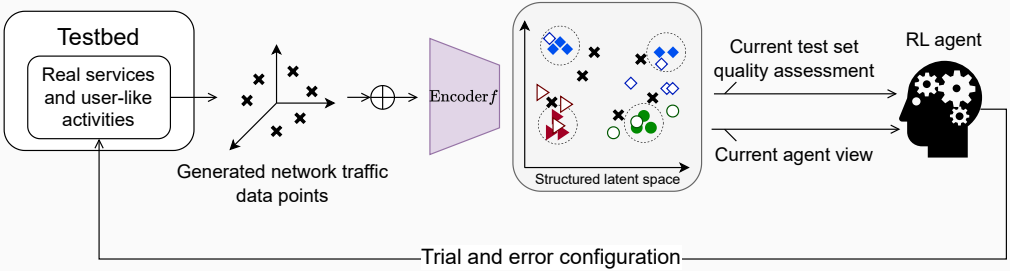
- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \boxed{\text{select a configuration}} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$

$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation



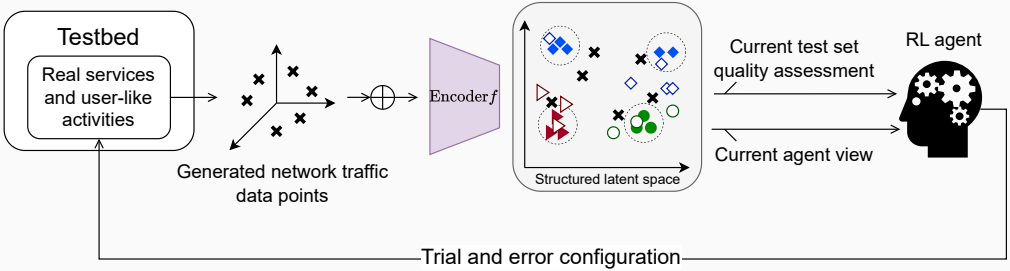
- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \boxed{\text{traffic generation}} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$

$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation



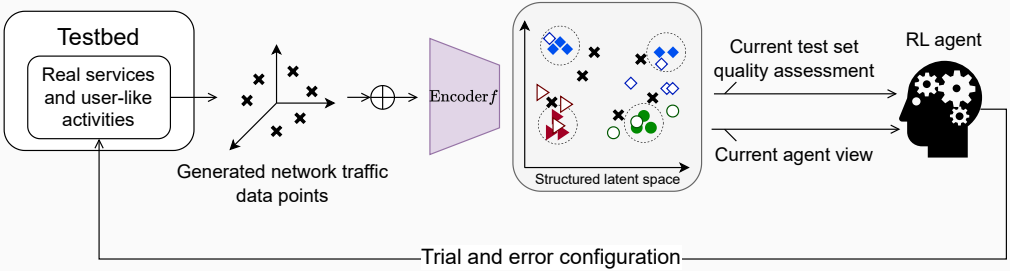
- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \boxed{\text{update } \mathcal{Z}_{\text{test}}}$$

$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation



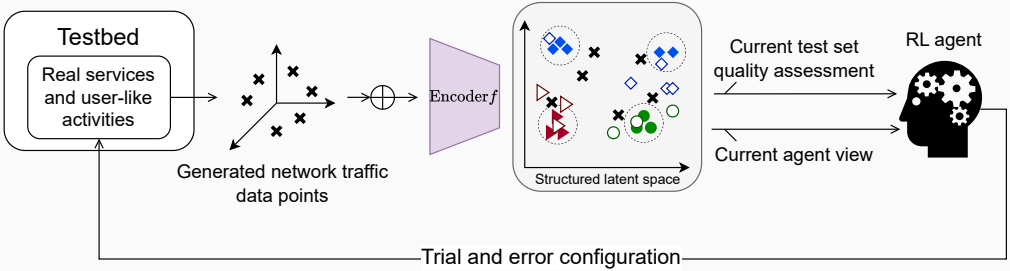
- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$

$$\rightarrow \boxed{\text{compute proximity, diversity, and scarcity}} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation

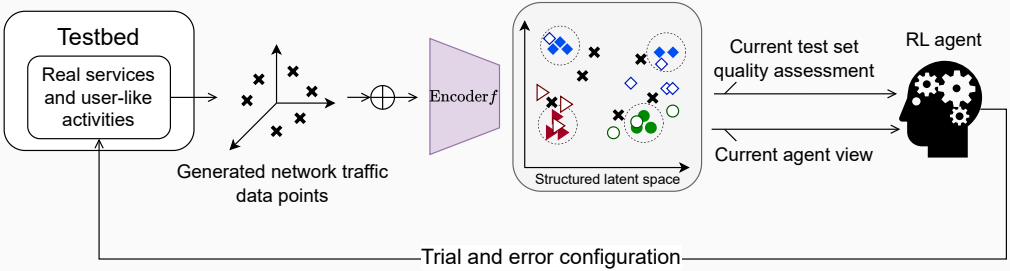


- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \boxed{\text{update policy } \pi} \circlearrowleft \Rightarrow \pi^*$$

Phase 2: Targeted Augmentation

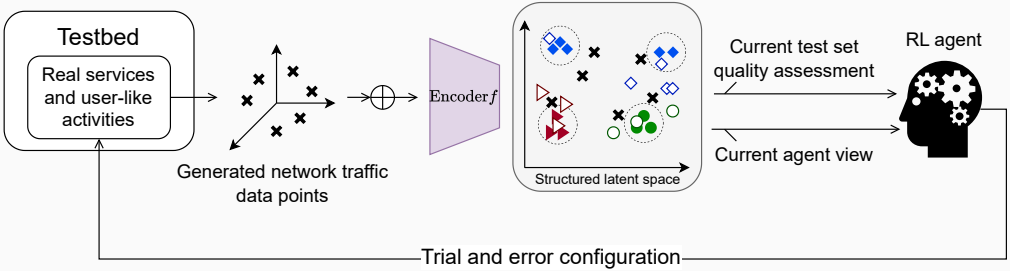


- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \; \boxed{\circlearrowright} \; \Rightarrow \; \pi^*$$

Phase 2: Targeted Augmentation



- Start from Preliminary & Phase 1 outputs
- Configurable live testbed for realistic traffic generation.
- Legitimate multi-protocol traffic (HTTP/HTTPS, SSH, SMTP, DNS, . . .) and controlled attack traffic.

- **Training workflow**:

$$\text{observe latent-space} \rightarrow \text{select a configuration} \rightarrow \text{traffic generation} \xrightarrow{\text{embed}} \text{update } \mathcal{Z}_{\text{test}}$$
$$\rightarrow \text{compute proximity, diversity, and scarcity} \rightarrow \text{update policy } \pi \circlearrowleft \Rightarrow \boxed{\pi^*}$$

Experimental Setup

Datasets (60 / 20 / 20 split)

(Using the revised and corrected versions)

- **CIC-IDS 2017**: 0.93 M benign / 0.25 M attack flows
- **CSE-CIC-IDS 2018**: 59.8 M benign / 4.1 M attack flows

NIDS Models

- RF, SVM, 3-layer DNN (128-128-128 ReLU + Softmax)

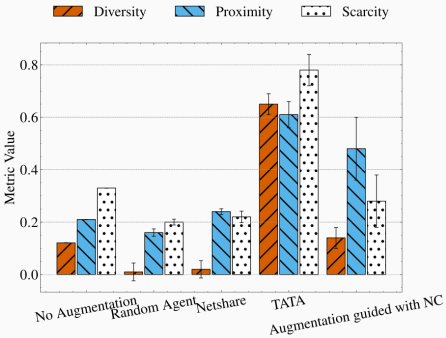
Agent Training

- Augmentation targets Benign SSH flows only

- Action space (client-side via `tc`): **Loss** $\in [5, 10]\%$ | **Jitter** $\in [4, 10]\,\mathrm{ms}$ | **Delay** $\in [10, 40]\,\mathrm{ms}$ | **Duplication** $\in [0.1, 5]\%$ | **Corruption** $\in [0.1, 10]\%$ | **Reordering** $\in [0.1, 50]\%$ | **Correlation** $\in [50, 100]\%$.

Test-set Augmentation — Metrics

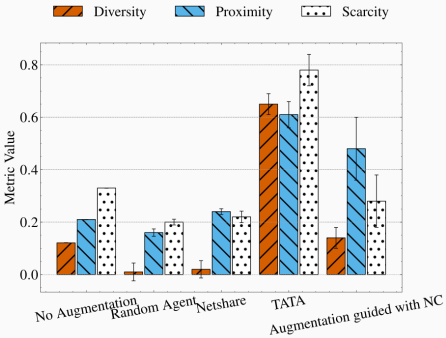
- **Protocol:** 20 eval episodes on IDS2017; no reward/updates; add Benign SSH flows; report mean±sd.
- **Baselines:** Random, NetShare (GAN), RL + Neuron Coverage (NC).



Test-set quality metrics (higher is more challenging/covered).

Test-set Augmentation — Metrics

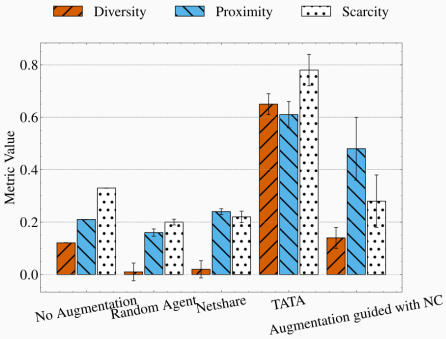
- **Protocol:** 20 eval episodes on IDS2017; no reward/updates; add Benign SSH flows; report mean±sd.
- **Baselines:** Random, NetShare (GAN), RL + Neuron Coverage (NC).
- **Quality metrics:**
  - **TATA:** $D + \sim 437\%$; $P + \sim 190\%$; $S + \sim 136\%$
  - **NC:** $D \approx 0$; $P + \sim 129\%$; $S \approx 0$
  - **Random/NetShare:** $D \downarrow$ to ≈ 0; $P \approx 0$; $S \downarrow$



Test-set quality metrics (higher is more challenging/covered).

Test-set Augmentation — Metrics

- **Protocol:** 20 eval episodes on IDS2017; no reward/updates; add Benign SSH flows; report mean±sd.
- **Baselines:** Random, NetShare (GAN), RL + Neuron Coverage (NC).
- **Quality metrics:**
  - **TATA:** $D + \sim 437\%$; $P + \sim 190\%$; $S + \sim 136\%$
  - **NC:** $D \approx 0$; $P + \sim 129\%$; $S \approx 0$
  - **Random/NetShare:** $D \downarrow$ to ≈ 0; $P \approx 0$; $S \downarrow$

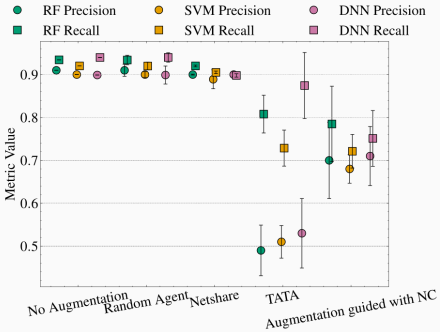


Test-set quality metrics (higher is more challenging/covered).

What is the impact on NIDS performance?

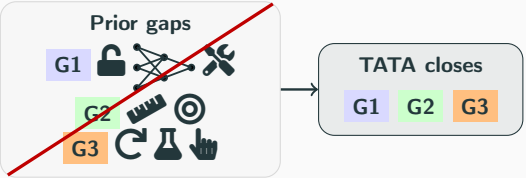
Test-set Augmentation — Impact on NIDS

- **NIDS impact (RF example):**
  - **TATA:** Precision $-\sim 46\%$; Recall $-\sim 13\%$
  - **NC:** Precision $-\sim 23\%$; Recall $-\sim 16\%$
- **Note:** higher scarcity spreads errors across boundaries $\Rightarrow$ FP $\uparrow \Rightarrow$ precision $\downarrow$.
- $\Rightarrow$ **proximity is the key driver;** scarcity and diversity are complementary (spread errors, avoid redundancy)

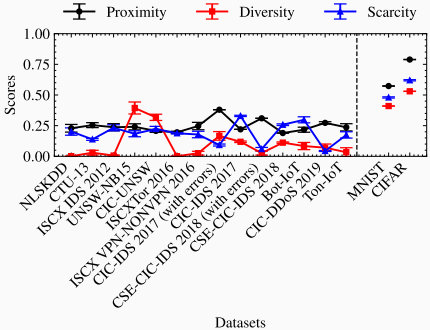


Impact on detection metrics (example: RF).

Part II conclusion



- Across **12 public NIDS datasets**, metrics remain mostly below **0.5** ⇒ limited test-set difficulty.



12 public NIDS datasets ordered on the x-axis by release year (2009–2020)

Conclusion and perspectives

Adaptation Part

Conclusion

- Re-running the search for a (sub-)optimal configuration after each context change is too expensive for adaptive ML-based NIDS.
- Meta-learning keeps reconfiguration fast while staying close to the (sub-)optimal configuration, including attack-free AD.

Perspectives

- Move toward continuous, closed-loop reconfiguration under network context change.
- Explore privacy-preserving and federated configuration learning to mitigate cold-start deployment.

Conclusion

- Static benchmark accuracy is not enough for robust NIDS evaluation.
- TATA assesses test-set difficulty and generates targeted, realistic raffic to expose hidden weaknesses.

Perspectives

- Reuse TATA-generated hard cases for detector hardening and retraining.
- Extend beyond the current flow-based, single-protocol setting toward richer traffic and larger testbeds.

Domain Perspectives

General takeaway

- Robust ML-based NIDS require both stronger pre-deployment evaluation and adaptive post-deployment configuration.

Perspectives

- Move from one-shot benchmarking to continuous evaluation and adaptation loops.
- Use GenAI for scenario-to-traffic generation and analyst assistance, while accounting for more adaptive attackers.

Questions?